Práctica 4 · Gestión de Versiones y Flujos de Trabajo con Git

Proyecto RTVRPTW · Laboratorio de Desarrollo y Herramientas

13 de noviembre de 2026

> **Estado del trabajo:** práctica reproducida y verificada **localmente** sobre una copia del proyecto de la Práctica 3. Pendiente de reproducir/publicar sobre el repositorio original de GitHub, ejecutar Maven/JUnit en el equipo y recoger las tres respuestas reales de GitHub Copilot. Este informe no presenta esos pasos como si ya se hubieran realizado.

1. Introducción y objetivo

El objetivo consiste en aplicar Git al proyecto RTVRPTW utilizado en la Práctica 3, creando ramas para una funcionalidad y una corrección, integrando ambos desarrollos, resolviendo un conflicto intencionado y etiquetando una versión estable. Se ha usado el código de RTVRPTW adaptado a Maven, con la documentación y correcciones de la práctica anterior.

Para disponer de evidencias verificables se construyó un **repositorio Git demostrativo local** a partir del proyecto de la Práctica 3. Como no se dispone de la URL ni del historial de commits del repositorio original de GitHub en los archivos facilitados, no se ha afirmado que se haya clonado ese repositorio ni se han ejecutado `push` remotos.

Repositorio GitHub actualizado: [pendiente de incorporar la URL del repositorio de la Práctica 3].

2. Configuración y ramas

La rama `main` se utilizó como línea base estable. Las ramas creadas desde ella fueron:

Rama	Finalidad	Resultado
feature/nueva-funcionalidad	Añadir un modo de consulta de instancias	Integrada en main y eliminada localmente
hotfix/correccion-sonarqube	Mejorar robustez de lectura de instancias	Integrada primero en main y eliminada localmente
test/conflicto	Provocar y resolver un conflicto de fusión	Resuelta; conservada como evidencia local

Se realizaron commits separados de cada modificación, en lugar de introducir todos los cambios directamente en `main`. Esto permite comprobar la evolución del código y el orden de integración.

3. Desarrollo en la rama `feature/nueva-funcionalidad`

Se añadió una función limitada y compatible con la arquitectura existente: **mostrar un resumen de una instancia TOPTW sin ejecutar GRASP**. Se implementó la clase `TOPTWInstanceSummary` y se amplió `mainTOPTW` para reconocer el argumento `--resumen`. También se incorporó `TOPTWInstanceSummaryTest` con una prueba sobre la instancia `c101Recortada.txt`.

Comando de uso (con Maven ya compilado):

```
java -jar target/rtvrptw-1.1.jar --resumen c101Recortada.txt
```

La prueba de ejecución directa local confirmó: 20 puntos de interés, 3 vehículos, horizonte 1236,0 y puntuación disponible de 360,0. No se modificó el algoritmo de optimización.

Commit local: a91198a · Feat: mostrar resumen de instancia sin ejecutar GRASP.

4. Corrección en hotfix/correccion-sonarqube

Se eligió una mejora de robustez asociada al área de fiabilidad de la Práctica 3: **manejo de instancias incompletas o mal formadas en TOPTWReader.readProblem**. Antes de la corrección, una línea inexistente o con pocas columnas podía producir una excepción poco explicativa. La nueva implementación verifica cabecera, cantidad de campos y presencia de líneas; conserva el cierre automático del lector con try-with-resources y comunica errores informativos.

Se añadieron dos pruebas JUnit: una para un archivo truncado y otra para una cabecera incompleta. La ejecución directa de comprobaciones equivalentes confirmó que ambos casos disparan la excepción esperada; la prueba automatizada con Maven queda pendiente del entorno local de la estudiante.

Commit local: 8640099 · Fix: validar ficheros TOPTW incompletos o mal formados.

Esta corrección es coherente con los problemas de fiabilidad estudiados mediante SonarQube, **pero no se atribuye a una regla exacta de SonarQube sin comprobarla**. Los resultados disponibles en la Práctica 3 fueron 6 incidencias de fiabilidad, 91 de mantenibilidad, 5,7 % de cobertura y un Quality Gate aprobado, medidos **antes** de estas modificaciones.

5. Integración de ramas

Se respetó el orden solicitado en el enunciado:

1. Fusionar el hotfix en main (f9c06c9).
2. Incorporar la nueva main a la rama feature (d82549e).
3. Fusionar la feature ya actualizada en main (5b4a47a).
4. Eliminar las ramas locales feature y hotfix, ya integradas.

Los comandos esenciales fueron:

```
git switch main
git merge --no-ff hotfix/correccion-sonarqube
git switch feature/nueva-funcionalidad
git merge --no-ff main
git switch main
git merge --no-ff feature/nueva-funcionalidad
git branch -d feature/nueva-funcionalidad
git branch -d hotfix/correccion-sonarqube
```

El historial registra commits de fusión explícitos, por lo que las ramas y su procedencia son visibles mediante git log --graph --oneline --decorate --all.

6. Simulación y resolución de un conflicto

Se creó test/conflicto desde main. Posteriormente se modificó la primera línea de README.md en cada rama, asignando títulos distintos. Tras registrar ambos commits, desde test/conflicto se ejecutó git merge main. Git detectó efectivamente CONFLICT (content): Merge conflict in README.md.

Fragmento real del archivo en conflicto:

```
<<<<<<< HEAD
# RTVRPTW · pruebas de integración (rama test)
=======
# RTVRPTW · versión 1.1 (rama principal)
>>>>>>> main
```

Se resolvió manualmente conservando un encabezado común: `# RTVRPTW · versión 1.1 — integración verificada`. Se eliminaron todos los marcadores, se ejecutaron `git add README.md` y `git commit`, y se integró la resolución en `main`.

Commits de evidencia: `444751e` (cambio en main), `b865f5f` (cambio en test), `51c0e45` (resolución), `8c732e3` (integración final). El ZIP incluye una copia de los marcadores originales en `conflicto_antes_de_resolver.txt`.

7. Etiquetado y sincronización

Se cambió la versión Maven a `1.1` y se creó el tag **anotado** `v1.1` sobre el commit local final `788542b`:

```
git tag -a v1.1 -m "RTVRPTW v1.1: resumen, hotfix y prácticas de integración"
git show --no-patch v1.1
```

En el repositorio original, y **solo después** de reproducir el flujo conservando su historial, los comandos de publicación son:

```
git push origin main
git push origin v1.1
```

No se han ejecutado esos `push` sobre GitHub. Subir la demostración local con `--force` sobre la rama remota anterior sería incorrecto porque reemplazaría el historial de la Práctica 3.

8. Uso de GitHub Copilot Chat

El enunciado exige tres interacciones reales con **GitHub Copilot Chat**. Los siguientes prompts están preparados para copiarlos en IntelliJ o VS Code. Las respuestas de Copilot **no se han obtenido ni simulado** en este entorno; deben incluirse después de ejecutarlos junto con las capturas y su valoración crítica.

8.1. Propuesta de funcionalidad (prompt preparado)

> Estoy trabajando en el proyecto Java RTVRPTW/TOPTW, construido con Maven. Sugiere una nueva funcionalidad sencilla y compatible con las clases `TOPTW`, `TOPTWReader` y `mainTOPTW`, sin modificar el algoritmo GRASP. Necesito clases afectadas, pseudocódigo, pruebas JUnit 5 y un comando de uso. Propongo un argumento `--resumen` para mostrar el número de puntos de interés, vehículos, límite de tiempo y puntuación total de una instancia. ¿Cómo lo implementarías de forma mínima?

Implementación para comparar: clase de resumen independiente, parámetro `--resumen`, una prueba JUnit. **Valoración técnica de referencia:** una solución adecuada debe evitar modificar el algoritmo y no duplicar el código de lectura; hay que contrastar estos criterios con la respuesta real.

8.2. Corrección del bug (prompt preparado)

> En el proyecto Java TOPTW, el método `TOPTWReader.readProblem(String)` lee instancias con `BufferedReader`. Una entrada truncada puede llegar a `null` antes de llamar a `simplifyString` o `split`, y una línea con pocas columnas causa un error poco descriptivo. Propón un hotfix mínimo que cierre siempre los recursos, valide cabecera, líneas y columnas, preserve el comportamiento con archivos correctos e incluya dos pruebas JUnit 5. No reformules el algoritmo de optimización.

Implementación para comparar: se comprobaron cabecera, filas y columnas, y se añadieron pruebas para entradas inválidas. **Valoración técnica de referencia:** evaluar si Copilot propone un control de errores claro sin refactorizaciones innecesarias. Indicar después si su propuesta fue aceptada total o parcialmente.

8.3. Conflicto de fusión (prompt preparado)

> He provocado un conflicto Git al modificar de forma diferente la primera línea de README.md en main y test/conflicto. Al ejecutar git merge main desde test/conflicto, veo <<<<<<< HEAD, ======= y >>>>>>> main. Explícame qué significa cada bloque, cómo resolverlo manualmente conservando un título unificado y qué comandos usar para concluir el merge. No me propongas git reset --hard ni git push --force.

Solución aplicada: edición manual del README, eliminación de los marcadores, git add y git commit.
Valoración técnica de referencia: comprobar si Copilot distingue las dos versiones y propone conservar la información útil sin descartar commits.

Pendiente indispensable: añadir debajo de cada prompt una síntesis fiel de la **respuesta real** de Copilot, una captura o transcripción y una comparación concreta con la solución implementada. No deben sustituirse por respuestas inventadas.

9. Verificaciones y limitaciones

Comprobación	Resultado
Historial Git, ramas y commits	Verificado en repositorio demostrativo local
Conflicto y resolución manual	Verificado; archivos de evidencia incluidos
Tag anotado v1.1 y árbol de trabajo limpio	Verificado localmente
Compilación Java 21 mediante javac	Correcta
Ejecución de --resumen y ejecución reducida de GRASP	Correctas
Comprobación directa de entradas truncadas/incompletas	Correcta
mvn clean verify con tests JUnit	Pendiente: Maven no instalado en este entorno
Capturas y respuestas reales de Copilot	Pendiente de uso en el IDE
URL GitHub y push de main/v1.1	Pendiente de publicación en el repositorio original

Se adjuntan registro_git_completo.txt, conflicto_antes_de_resolver.txt, parches separados de las dos ramas y un ZIP con el repositorio Git local para consulta. Las comprobaciones locales son reales, pero no sustituyen las evidencias de GitHub y Copilot solicitadas por el docente.

10. Reflexión final

Git permite conservar una línea estable mientras se realizan cambios aislados y revisables. Las ramas de funcionalidad y de corrección evitan mezclar tareas diferentes; los commits describen cada paso y los merges registran el modo en que se integraron. El conflicto de README mostró que dos cambios válidos pueden ser incompatibles y que es necesario decidir cómo combinarlos, en lugar de sobrescribir silenciosamente uno de ellos. El tag v1.1 identifica un estado concreto del proyecto.

Sin un sistema de control de versiones sería mucho más difícil conocer qué se cambió, recuperar una versión anterior y colaborar sin perder trabajo. GitHub Copilot puede ayudar a proponer código o interpretar conflictos, pero sus indicaciones deben contrastarse con el proyecto, las pruebas y el historial real de Git. Para una entrega académica rigurosa, la trazabilidad y la verificación de lo realizado son tan importantes como la implementación.